

SQL Server

Common Table Expressions (CTEs)

&

Subqueries

A Complete End-to-End Reference Guide

Covers: Syntax • Types • Recursive CTEs • Performance • Best Practices

1. Introduction

SQL Server provides two powerful mechanisms for breaking complex queries into manageable, readable pieces: Subqueries and Common Table Expressions (CTEs). Understanding when and how to use each one is a core competency for any SQL Server developer or DBA.

This guide covers everything from the fundamentals to advanced recursive patterns, with performance considerations and real-world examples throughout.

1.1 What Is a Subquery?

A subquery (also called an inner query or nested query) is a `SELECT` statement embedded inside another SQL statement. The outer statement uses the result of the inner query as part of its own logic.

**KEY
POINT**

Subqueries are evaluated from the innermost query outward. The database engine resolves the deepest nested `SELECT` first, then passes its result set up to the parent query.

1.2 What Is a CTE?

A Common Table Expression (CTE) is a named, temporary result set that exists only for the duration of a single SQL statement. Defined with the `WITH` keyword, CTEs improve readability, allow self-referencing (recursive CTEs), and can be referenced multiple times within the same query.

**KEY
POINT**

Unlike a subquery, a CTE is defined once at the top of the statement and can be referenced by name multiple times within that statement — making complex logic far easier to write and maintain.

2. Subqueries

2.1 Syntax & Placement

A subquery must always be enclosed in parentheses. It can appear in four positions within an outer statement:

- **SELECT clause** (scalar subquery — returns exactly one row/column)
- **FROM clause** (derived table / inline view)
- **WHERE clause** (filter using a value or set returned by the inner query)
- **HAVING clause** (filter on aggregated values)

2.1.1 Scalar Subquery in SELECT

Returns a single value per row of the outer query. Useful for adding a calculated column without a JOIN.

```
-- Scalar subquery: attach the department name to each employee
SELECT
    e.EmployeeID,
    e.FirstName,
    e.LastName,
    (SELECT d.DepartmentName
     FROM   dbo.Departments d
     WHERE  d.DepartmentID = e.DepartmentID) AS DepartmentName
FROM dbo.Employees e;
```



A scalar subquery that returns more than one row causes a runtime error: 'Subquery returned more than 1 value.' Always ensure the inner query returns at most one row.

2.1.2 Derived Table in FROM

A subquery in the FROM clause acts as an inline view (derived table). You must give it an alias.

```
-- Derived table: average salary per department, then filter departments
SELECT
    dept_avg.DepartmentID,
    dept_avg.DeptName,
    dept_avg.AvgSalary
FROM (
    SELECT
        d.DepartmentID,
        d.DepartmentName AS DeptName,
        AVG(e.Salary)     AS AvgSalary
    FROM   dbo.Employees e
    GROUP BY d.DepartmentID, d.DepartmentName
)
```

```
FROM    dbo.Employees e
JOIN    dbo.Departments d ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
) AS dept_avg
WHERE dept_avg.AvgSalary > 70000;
```

2.1.3 Subquery in WHERE (Non-Correlated)

A non-correlated subquery is independent of the outer query. It executes once and its result is used by the outer WHERE filter.

```
-- Find employees in departments located in 'New York'
SELECT e.EmployeeID, e.FirstName, e.LastName
FROM   dbo.Employees e
WHERE  e.DepartmentID IN (
    SELECT DepartmentID
    FROM   dbo.Departments
    WHERE  Location = 'New York'
);
```

2.1.4 Correlated Subquery in WHERE

A correlated subquery references a column from the outer query. It executes once per row of the outer query — making it powerful but potentially expensive on large tables.

```
-- Find employees whose salary is above their own department's average
SELECT e.EmployeeID, e.FirstName, e.LastName, e.Salary
FROM   dbo.Employees e
WHERE  e.Salary > (
    SELECT AVG(e2.Salary)
    FROM   dbo.Employees e2
    WHERE  e2.DepartmentID = e.DepartmentID  -- correlated reference
);
```

PERFORMANCE

Correlated subqueries execute once per outer row. For large tables, rewrite as a JOIN against a derived table or CTE to allow the optimizer to compute the aggregate once.

2.2 EXISTS and NOT EXISTS

EXISTS tests whether a subquery returns any rows. It short-circuits as soon as the first matching row is found — often faster than IN when the inner set is large.

```
-- EXISTS: employees who have placed at least one order
SELECT e.EmployeeID, e.FirstName, e.LastName
FROM   dbo.Employees e
```

```
WHERE EXISTS (
    SELECT 1
    FROM    dbo.Orders o
    WHERE   o.SalespersonID = e.EmployeeID
);

-- NOT EXISTS: employees with no orders at all
SELECT e.EmployeeID, e.FirstName, e.LastName
FROM    dbo.Employees e
WHERE   NOT EXISTS (
    SELECT 1
    FROM    dbo.Orders o
    WHERE   o.SalespersonID = e.EmployeeID
);
```

2.3 ALL, ANY / SOME

These operators compare a value against every value returned by a subquery.

```
-- ALL: employees whose salary exceeds EVERY salary in department 10
SELECT EmployeeID, FirstName, Salary
FROM    dbo.Employees
WHERE   Salary > ALL (
    SELECT Salary FROM dbo.Employees WHERE DepartmentID = 10
);

-- ANY: employees who earn more than AT LEAST ONE person in dept 10
SELECT EmployeeID, FirstName, Salary
FROM    dbo.Employees
WHERE   Salary > ANY (
    SELECT Salary FROM dbo.Employees WHERE DepartmentID = 10
);
```

3. Common Table Expressions (CTEs)

3.1 Basic Syntax

The WITH keyword opens a CTE definition. Multiple CTEs can be chained in one statement, separated by commas. The final SELECT (or DML) statement uses the named result sets.

```
-- Full CTE syntax template
WITH cte_name1 (Col1, Col2, ...)  -- optional column list
AS (
    -- CTE query definition
    SELECT ...
),
cte_name2 AS (                    -- second CTE (no WITH keyword)
    SELECT ...
    FROM   cte_name1              -- can reference prior CTEs
)
-- Final statement using the CTEs
SELECT *
FROM   cte_name2;
```

SYNTAX RULE

If there is any statement before the WITH keyword (other than a semicolon), SQL Server will raise a syntax error. Always terminate the preceding statement with a semicolon, or start a fresh batch.

3.2 Single CTE Example

```
-- CTE: calculate department averages, then use in the main query
;WITH DeptAvg AS (
    SELECT
        DepartmentID,
        AVG(Salary) AS AvgSalary,
        COUNT(*)   AS HeadCount
    FROM   dbo.Employees
    GROUP BY DepartmentID
)
SELECT
    e.FirstName,
    e.LastName,
    e.Salary,
    da.AvgSalary AS DeptAvgSalary,
    e.Salary - da.AvgSalary AS DiffFromAvg
FROM   dbo.Employees e
JOIN   DeptAvg da ON da.DepartmentID = e.DepartmentID
ORDER BY DiffFromAvg DESC;
```

3.3 Multiple Chained CTEs

Chain as many CTEs as needed by separating them with a comma. Each CTE can reference any CTE defined before it.

```
;WITH
-- Step 1: orders in the last 90 days
RecentOrders AS (
    SELECT OrderID, CustomerID, OrderDate, TotalAmount
    FROM    dbo.Orders
    WHERE   OrderDate >= DATEADD(DAY, -90, GETDATE())
),
-- Step 2: aggregate per customer
CustomerTotals AS (
    SELECT CustomerID,
           COUNT(*)      AS OrderCount,
           SUM(TotalAmount) AS TotalSpend
    FROM    RecentOrders
    GROUP BY CustomerID
),
-- Step 3: rank customers by spend
RankedCustomers AS (
    SELECT CustomerID, OrderCount, TotalSpend,
           RANK() OVER (ORDER BY TotalSpend DESC) AS SpendRank
    FROM    CustomerTotals
)
-- Final: top 10 customers
SELECT TOP 10
    c.CustomerName,
    rc.OrderCount,
    rc.TotalSpend,
    rc.SpendRank
FROM    RankedCustomers rc
JOIN    dbo.Customers   c ON c.CustomerID = rc.CustomerID
ORDER BY rc.SpendRank;
```

3.4 CTE with DML (INSERT / UPDATE / DELETE)

CTEs are not limited to SELECT statements. You can use them with INSERT, UPDATE, and DELETE to stage transformation logic before modifying data.

3.4.1 UPDATE via CTE

```
;WITH SalaryAdj AS (
    SELECT
        e.EmployeeID,
        e.Salary,
        e.Salary * 1.10 AS NewSalary
    FROM    dbo.Employees e
    JOIN    dbo.Departments d ON d.DepartmentID = e.DepartmentID
    WHERE   d.DepartmentName = 'Engineering'
    AND     e.HireDate < DATEADD(YEAR, -3, GETDATE())
)
```

```
)  
UPDATE SalaryAdj  
SET     Salary = NewSalary;
```

3.4.2 DELETE via CTE

```
-- Remove duplicate rows, keeping the one with the lowest ID  
;WITH Dupes AS (  
    SELECT  
        ROW_NUMBER() OVER (  
            PARTITION BY Email  
            ORDER BY     CustomerID  
        ) AS RowNum,  
        CustomerID  
    FROM dbo.Customers  
)  
DELETE FROM Dupes  
WHERE RowNum > 1;
```

4. Recursive CTEs

A recursive CTE references itself. It consists of two parts joined by UNION ALL:

- **Anchor member** — the base case (non-recursive SELECT)
- **Recursive member** — references the CTE name, adding one level of recursion per iteration

SQL Server terminates recursion when the recursive member returns no rows, or when the MAXRECURSION limit is reached (default: 100).

4.1 Syntax

```
;WITH RecursiveCTE (Col1, Col2, Level)
AS (
    -- Anchor: starting rows
    SELECT Col1, Col2, 0 AS Level
    FROM   SourceTable
    WHERE  <base condition>

    UNION ALL

    -- Recursive: join CTE back to source to get next level
    SELECT s.Col1, s.Col2, r.Level + 1
    FROM   SourceTable s
    JOIN   RecursiveCTE r ON r.Col2 = s.ParentCol
    WHERE  r.Level < 10    -- optional guard
)
SELECT * FROM RecursiveCTE
OPTION (MAXRECURSION 500); -- override default of 100 if needed
```

4.2 Employee Hierarchy (Manager Tree)

```
-- Employees table: EmployeeID, Name, ManagerID (NULL for top-level)

;WITH OrgChart AS (
    -- Anchor: CEO / top-level employees (no manager)
    SELECT
        EmployeeID,
        Name,
        ManagerID,
        0          AS Level,
        CAST(Name AS NVARCHAR(1000)) AS HierarchyPath
    FROM   dbo.Employees
    WHERE  ManagerID IS NULL
```

```
UNION ALL

-- Recursive: direct reports of the current level
SELECT
    e.EmployeeID,
    e.Name,
    e.ManagerID,
    oc.Level + 1,
    CAST(oc.HierarchyPath + ' > ' + e.Name AS NVARCHAR(1000))
FROM dbo.Employees e
JOIN OrgChart      oc ON oc.EmployeeID = e.ManagerID
)
SELECT
    REPLICATE(' ', Level) + Name AS IndentedName,
    Level,
    HierarchyPath
FROM OrgChart
ORDER BY HierarchyPath;
```

4.3 Date Series Generator

Recursive CTEs are excellent for generating sequences of dates or numbers without needing a helper table.

```
-- Generate every date in a range
DECLARE @StartDate DATE = '2024-01-01';
DECLARE @EndDate   DATE = '2024-12-31';

;WITH DateSeries AS (
    SELECT @StartDate AS CalDate

    UNION ALL

    SELECT DATEADD(DAY, 1, CalDate)
    FROM   DateSeries
    WHERE  DATEADD(DAY, 1, CalDate) <= @EndDate
)
SELECT
    CalDate,
    DATENAME(WEEKDAY, CalDate) AS DayName,
    DATEPART(WEEK, CalDate) AS WeekNum
FROM DateSeries
OPTION (MAXRECURSION 366);
```

5. CTEs vs Subqueries — Side-by-Side

Feature	Subquery	CTE
Syntax placement	Inline within SELECT, FROM, WHERE, HAVING	Declared above the main statement with WITH
Reusability in same query	Must be duplicated each time	Defined once, referenced by name multiple times
Readability	Can become hard to parse when nested deeply	Linear, named steps improve clarity
Recursion support	Not supported	Fully supported (recursive CTEs)
DML support	Limited (derived tables only)	Yes — CTE can be the target of UPDATE/DELETE
Materialization	Inlined by optimizer (usually)	Inlined by default; use MATERIALIZE hint if needed
Scope	Local to the expression it appears in	Available to entire statement below its definition
Indexing	Not directly indexable	Not directly indexable (use temp tables if needed)
Multiple references	Re-executed each time it appears	May be re-evaluated each reference (inlined)
SQL Server version	All versions	SQL Server 2005+

5.1 Same Query: Subquery vs CTE

The following two queries are semantically identical. Compare readability:

Subquery Version

```
SELECT
    dept_data.DepartmentName,
    dept_data.AvgSalary,
    top_emp.TopEarner
FROM (
    SELECT d.DepartmentName, AVG(e.Salary) AS AvgSalary
    FROM   dbo.Employees e
    JOIN   dbo.Departments d ON d.DepartmentID = e.DepartmentID
    GROUP BY d.DepartmentName
) dept_data
JOIN (
    SELECT d.DepartmentName, MAX(e.Salary) AS TopEarner
    FROM   dbo.Employees e
    JOIN   dbo.Departments d ON d.DepartmentID = e.DepartmentID
```

```
GROUP BY d.DepartmentName
) top_emp ON top_emp.DepartmentName = dept_data.DepartmentName;
```

CTE Version (preferred)

```
;WITH
DeptStats AS (
    SELECT
        d.DepartmentName,
        AVG(e.Salary) AS AvgSalary,
        MAX(e.Salary) AS TopEarner
    FROM    dbo.Employees e
    JOIN    dbo.Departments d ON d.DepartmentID = e.DepartmentID
    GROUP BY d.DepartmentName
)
SELECT DepartmentName, AvgSalary, TopEarner
FROM    DeptStats;
```

BEST PRACTICE

When the same derived table would appear more than once, or when nesting exceeds two levels, refactor to a CTE. The optimizer typically generates the same execution plan; the gain is developer productivity and maintainability.

6. Performance Considerations

6.1 How SQL Server Processes CTEs and Subqueries

By default, SQL Server inlines both CTEs and subqueries into the outer query before optimization. This means the query optimizer sees them as one unified expression — there is no guaranteed intermediate materialization.

- A CTE referenced twice may be executed twice (it is not automatically cached).
- Use a #temp table or table variable when you need guaranteed one-time execution of expensive logic.
- The WITH RESULT SETS or OPTION (RECOMPILE) hints can help in edge cases.

6.2 Correlated Subquery vs JOIN Performance

```
-- SLOW: correlated subquery (executes once per outer row)
SELECT e.EmployeeID, e.Salary,
       (SELECT AVG(e2.Salary)
        FROM   dbo.Employees e2
        WHERE  e2.DepartmentID = e.DepartmentID) AS DeptAvg
FROM   dbo.Employees e;

-- FAST: pre-aggregate with CTE, then JOIN
;WITH DeptAvg AS (
    SELECT DepartmentID, AVG(Salary) AS AvgSalary
    FROM   dbo.Employees
    GROUP BY DepartmentID
)
SELECT e.EmployeeID, e.Salary, da.AvgSalary AS DeptAvg
FROM   dbo.Employees e
JOIN   DeptAvg da ON da.DepartmentID = e.DepartmentID;
```

6.3 When to Use Temp Tables Instead

If a CTE is referenced multiple times and contains expensive logic, materialize it into a temporary table:

```
-- Materialize expensive CTE logic into a temp table
SELECT
    DepartmentID,
    AVG(Salary) AS AvgSalary,
    MAX(Salary) AS MaxSalary,
    COUNT(*) AS HeadCount
INTO #DeptStats
FROM   dbo.Employees
```

```
GROUP BY DepartmentID;

CREATE INDEX IX_DeptStats_DeptID ON #DeptStats (DepartmentID);

-- Now reference #DeptStats multiple times cheaply
SELECT e.*, ds.AvgSalary
FROM   dbo.Employees e
JOIN   #DeptStats ds ON ds.DepartmentID = e.DepartmentID;

DROP TABLE #DeptStats;
```

6.4 MAXRECURSION Control

```
-- Default: 100 levels. Set to 0 for unlimited (use with caution!).
;WITH Numbers AS (
    SELECT 1 AS N
    UNION ALL
    SELECT N + 1 FROM Numbers WHERE N < 1000
)
SELECT N FROM Numbers
OPTION (MAXRECURSION 1000);

-- MAXRECURSION 0 = unlimited (ensure termination condition is solid)
OPTION (MAXRECURSION 0);
```

7. Advanced Patterns

7.1 Running Totals with CTEs

```
;WITH OrderedSales AS (  
    SELECT  
        OrderDate,  
        TotalAmount,  
        SUM(TotalAmount) OVER (  
            ORDER BY OrderDate  
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
        ) AS RunningTotal  
    FROM dbo.Orders  
)  
SELECT OrderDate, TotalAmount, RunningTotal  
FROM OrderedSales  
ORDER BY OrderDate;
```

7.2 Pagination with CTE + ROW_NUMBER

```
DECLARE @PageNum INT = 3;  
DECLARE @PageSize INT = 20;  
  
;WITH PagedResults AS (  
    SELECT  
        EmployeeID,  
        FirstName,  
        LastName,  
        Salary,  
        ROW_NUMBER() OVER (ORDER BY LastName, FirstName) AS RowNum  
    FROM dbo.Employees  
)  
SELECT EmployeeID, FirstName, LastName, Salary  
FROM PagedResults  
WHERE RowNum BETWEEN (@PageNum - 1) * @PageSize + 1  
    AND @PageNum * @PageSize  
ORDER BY RowNum;
```

7.3 MERGE Statement with CTE

```
-- Stage incoming data in a CTE, then MERGE into target  
;WITH Source AS (  
    SELECT ProductID, ProductName, Price  
    FROM dbo.StagingProducts  
    WHERE LoadDate = CAST(GETDATE() AS DATE)  
)  
MERGE dbo.Products AS Target
```

```
USING Source ON Target.ProductID = Source.ProductID
WHEN MATCHED THEN
    UPDATE SET
        Target.ProductName = Source.ProductName,
        Target.Price       = Source.Price
WHEN NOT MATCHED BY TARGET THEN
    INSERT (ProductID, ProductName, Price)
    VALUES (Source.ProductID, Source.ProductName, Source.Price)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

7.4 Multi-Level Hierarchy Flattening

```
-- Flatten an org chart into a path string and depth column
;WITH Hierarchy AS (
    SELECT
        EmployeeID,
        Name,
        ManagerID,
        1 AS Depth,
        CAST(EmployeeID AS VARCHAR(MAX)) AS IDPath
    FROM dbo.Employees
    WHERE ManagerID IS NULL

    UNION ALL

    SELECT
        e.EmployeeID,
        e.Name,
        e.ManagerID,
        h.Depth + 1,
        h.IDPath + '.' + CAST(e.EmployeeID AS VARCHAR(20))
    FROM dbo.Employees e
    JOIN Hierarchy h ON h.EmployeeID = e.ManagerID
)
SELECT
    EmployeeID,
    Name,
    Depth,
    IDPath
FROM Hierarchy
ORDER BY IDPath;
```

8. Best Practices & Common Pitfalls

8.1 Best Practices

1. Always prefix CTEs with a semicolon (;)WITH) to guard against missing terminators on the previous statement.
2. Give CTEs descriptive names that communicate their purpose (e.g., ActiveCustomers, DeptSalaryStats).
3. Keep each CTE focused on a single transformation step. Chaining three focused CTEs beats one monolithic query.
4. Prefer EXISTS over IN when filtering with a subquery against a large inner set — EXISTS short-circuits.
5. Specify the MAXRECURSION option explicitly for recursive CTEs so behavior is predictable and documented.
6. For correlated subqueries that run per-row, consider rewriting as a pre-aggregated CTE joined back to the main table.
7. If a CTE is referenced more than once and is expensive, materialize it into a #temp table with an appropriate index.

8.2 Common Pitfalls

**PITFALL
1**

Forgetting that CTEs are not persisted. A CTE exists only for the duration of the single statement it belongs to. You cannot reference it from a second statement or stored procedure call.

**PITFALL
2**

Assuming a CTE is always materialized. SQL Server may inline the CTE multiple times when it is referenced more than once. Use #temp tables for guaranteed single execution.

**PITFALL
3**

Hitting the MAXRECURSION limit. The default of 100 is often too low for real hierarchies or date series. Always set OPTION (MAXRECURSION n) explicitly.

**PITFALL
4**

Using ORDER BY inside a CTE or subquery without TOP, OFFSET, or FOR XML. SQL Server does not guarantee row order within a CTE. Sorting belongs in the outermost query.

**PITFALL
5**

Nesting subqueries too deeply. Beyond two or three levels, deeply nested subqueries become very hard to debug. Refactor to CTEs for clarity.

9. Quick Reference Card

Scenario	Recommended Approach
Simple one-off inline filter	Subquery in WHERE with IN or EXISTS
Single-level derived table used once	Subquery in FROM (derived table)
Derived table referenced multiple times	CTE
Deeply nested or multi-step transformations	Chained CTEs
Hierarchical / tree data traversal	Recursive CTE
Row deduplication before DELETE	CTE targeting a DELETE
Conditional UPDATE using calculated columns	CTE targeting an UPDATE
Running totals / window functions	CTE + window function
Pagination	CTE + ROW_NUMBER()
Expensive logic referenced multiple times	#Temp table
Generate a number/date series	Recursive CTE with MAXRECURSION
Check for existence without fetching data	EXISTS subquery

9.1 Syntax Cheat Sheet

```
-- — Subquery Types —————
SELECT (SELECT scalar FROM t WHERE ...)           -- scalar in SELECT
FROM   (SELECT ... FROM t) AS alias                -- derived table
WHERE  col IN (SELECT col FROM t WHERE ...)         -- non-correlated
WHERE  col > (SELECT agg FROM t WHERE t.x = o.x)    -- correlated
WHERE  EXISTS (SELECT 1 FROM t WHERE t.x = o.x)     -- EXISTS

-- — CTE Skeleton —————
;WITH cte1 AS (SELECT ...),
      cte2 AS (SELECT ... FROM cte1)
SELECT * FROM cte2;

-- — Recursive CTE Skeleton —————
;WITH rcte AS (
    SELECT ...                                     -- anchor
    UNION ALL
    SELECT ... FROM src JOIN rcte ...               -- recursive member
)
SELECT * FROM rcte
OPTION (MAXRECURSION n);

-- — CTE + DML —————
;WITH cte AS (SELECT ... FROM target WHERE ...)
```

```
UPDATE cte SET col = val;  
  
;WITH cte AS (SELECT ..., ROW_NUMBER() OVER (...) AS rn FROM t)  
DELETE FROM cte WHERE rn > 1;
```